

Contents

1	week 3: linear regression	1
1.1	overview of Linear Regression	1
1.1.1	"linear" relationships	1
1.1.2	"supervised learning"	2
1.1.3	defintion	2
1.1.4	graphing linear regresssion	2
1.1.5	3 formulas under the hood	3
1.2	scikit learn	4
1.2.1	loading, prepping data	4
1.2.2	training the model	6
1.2.3	evaluating performance	7
1.2.4	multiple linear regression	8
1.3	break	9
1.4	logistic regression	9
1.5	notes	9

1 week 3: linear regression

- intuit linear regression
- hypothesis, loss, gradient descent
- practice with sci-kit learn
- break
- intuit logistic regression
- practice with sci-kit learn
- time to practice on own datasets

1.1 overview of Linear Regression

1.1.1 "linear" relationships

What is the relationship between the three columns, going row by row?
What is the pattern hidden in each row?

a	b	c
4	2	8
1	2	5
0	5	10
2	1	4

$$a^2 + b = c$$

This is what's called a linear relationship.

1.1.2 "supervised learning"

When the data is labeled in some way. We will see examples of learning where the data is not labeled at all, next week.

1.1.3 definition

Linear Regression is a supervised learning algorithm that predicts continuous values based on one or more input variables. It fits a straight line that best represents the data.

The goal of linear regression is to find a straight line that *minimizes the error* (the difference) between the observed data points and the predicted values. This line helps us predict the dependent variable for new, unseen data.

1.1.4 graphing linear regression

Housing example:

- you have a dataset of house size and their prices.
- the larger the house, the higher the price
- linear regression wants to find the line that cuts through the most points as possible
- the size of the houses is the X
 - input variable
- the price is the Y
 - target variable -> what we want to predict
- (x, y) is a single training example

1.1.5 3 formulas under the hood

You want Y to represent a straight line.

- What is the formula for a line?
- w, b are coefficients, parameters.

$$Y = mx + b$$

or,

$$f(x) = wx + b$$

Some intuition:

- $f(x) = 0 \cdot x + 1.5$
 - $w = 0, b = 1.5$
 - the line starts at $y = 1.5$ and runs horizontal (flat) across the x axis)
- $f(x) = 0.5 \cdot x + 0$
 - $w = .5$ (which is really $.5/1$)
 - $b = 0$
- $f(x) = .5 \cdot x + 1$
 - is actually the same line as above, but starts at 1

The goal, as you can see above, is to have right values for w, b so that you can fit a straight line best through the data.

So how do you find these values for w, b ?

- we need to measure how well our line fits our data.
- you start with the cost function:
 - measures the error between our prediction and the actual value
 - MSE - mean squared error (squared to get rid of negatives)

$$(y(\text{hat}) - y^i)^2$$

The cost function takes J as its variable.

- you want to get the smallest J possible.
- you want to minimize the error between the prediction and the actual value.
- the J graph is actually a parabola – there is a lowest point for J, and that is what we want.
 - we input different values of w until we get the lowest value for J.
 - see Andrew Ng, #12 Machine Learning Specialization [Course 1, Week 1, Lesson 3]

But how do we know what values to pick for w that will minimize J?

- you use the gradient descent algorithm
- gradient descent finds where to minimize w.
 - the formula takes current w and updates it by subtracting a "derivative" of w from w.

$w = w - \text{derivative of } w$

Once you get the lowest possible value for w, then you plug it back into the line function, and you're done!

1.2 scikit learn

From datacamp: <https://www.datacamp.com/tutorial/sklearn-linear-regression>

1.2.1 loading, prepping data

conda activate 656-ml conda install matplotlib seaborn

Importing libraries, loading data

```
# Import libraries.
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Let's use the well-known California housing dataset.
```

```
# Read in California housing dataset.
from sklearn.datasets import fetch_california_housing
```

```
housing = fetch_california_housing()
```

Preparing the data

- Let's split the data into training and testing sets.
- We import the `train_test_split()` method from `sklearn.model_selection`, then invoke it, specifying a test set percentage, and a `random_state`.
- We'll also use simple linear regression, using the feature corresponding to the average number of rooms.

```
# Import train_test_split.
from sklearn.model_selection import train_test_split
```

```
# Create features X and target y.
X = pd.DataFrame(housing.data, columns=housing.feature_names)[["AveRooms"]]
y = housing.target # Median house value in $100,000s
```

```
# Split the dataset into training (80%) and testing (20%) sets.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=
```

```
# Examine our training data
```

```
X_train
```

Now that we've split the data into test and train sets, let's standardize features. This process ensures that all variables are on the same scale, which can improve model performance and numerical stability.

```
# Import StandardScaler.
from sklearn.preprocessing import StandardScaler
```

```
# Instantiate StandardScaler.
scaler = StandardScaler()
```

```
# Fit and transform training data.
X_train_scaled = scaler.fit_transform(X_train)

# Also transform test data.
X_test_scaled = scaler.transform(X_test)
```

In this code, StandardScaler is a data preprocessing tool used to remove the mean and scale features to unit variance. This helps prevent certain features from dominating the model due to differences in scale.

- Compare `X_train` with `X_train_scaled` to see the difference.

1.2.2 training the model

Now, we will train the linear regression model

- To create a linear regression model, import `LinearRegression()` from `sklearn.linear_model`. Invoke it and assign it to a variable.

```
# Import LinearRegression.
from sklearn.linear_model import LinearRegression

# Instantiate linear regression model.
model = LinearRegression()

# Fit the model to the training data.
model.fit(X_train_scaled, y_train)
```

See coefficients:

- this is the weight, the "w", which is multiplied by all of the inputs in the x column.

```
model.coef_
```

Making predictions

- Now that we've trained our model, we make predictions on the test set.

```
# Make predictions on the testing data.
y_pred = model.predict(X_test_scaled)

# see our predictions
y_pred
```

1.2.3 evaluating performance

Now that we've made predictions on the test set, we need to know how well they match with reality. There are several metrics available to evaluate the performance of a regression algorithm. Some of the most common ones are the coefficient of determination (R^2), mean squared error (MSE), and root mean squared error (RMSE).

The coefficient of determination, denoted R^2 , measures how well a regression model explains the variability of the target variable. In other words, it quantifies how much of the variability in the target variable is explained by the predictors, known as the goodness of fit.

How do we interpret R^2 ?

$R^2 = 1$: the model perfectly explains all the variance in the target variable.

$R^2 = 0$: the model explains none of the variance; predictions are no better than simply using the mean.

$R^2 < 0$: The model performs worse than simply using the mean, indicating a poor fit.

```
# Import metrics.
from sklearn.metrics import mean_squared_error, r2_score
```

```
# Calculate and print  $R^2$  score.
r2 = r2_score(y_test, y_pred)
print(f"R-squared: {r2:.4f}")
```

MSE calculates the average squared difference between actual and predicted values.

```
# Calculate and print MSE.
mse = mean_squared_error(y_test, y_pred)
print(f"Mean squared error: {mse:.4f}")
```

1.2.4 multiple linear regression

Let's rerun the model using all of our available features, not just the average number of rooms. Do you expect better or worse results?

```
# Uses all features.
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# Load data set.
housing = fetch_california_housing()

# Split into X, y.
X = pd.DataFrame(housing.data, columns=housing.feature_names)
y = housing.target # Median house value in $100,000s
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scale the data.
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Create model and fit it to the training data.
model = LinearRegression()
model.fit(X_train_scaled, y_train)

# Make predictions.
```



```

y_pred = model.predict(X_test_scaled)

# Calculate and print errors.
r2 = r2_score(y_test, y_pred)
print(f"R-squared: {r2:.4f}")

mse = mean_squared_error(y_test, y_pred)
print(f"Mean squared error: {mse:.4f}")

rmse = mse ** 0.5
print(f"Root mean squared error: {rmse:.4f}")

```

We see the results are quite a bit better than when using only one feature. However, this raises the question of whether we need all the features. Are some features more relevant than others? Choosing the most relevant features from the dataset is known as feature selection.

Feature selection is important for a number of reasons.

- Improves accuracy. Removing irrelevant or redundant features helps the model focus on meaningful patterns.
- Speeds up training. Reducing the number of features decreases computational time and memory usage.

1.3 break

1.4 logistic regression

- wants to predict categories, classes, categorical values.

1.5 notes

Linear regression formula:

$$y(w,x) = w_0 + w_1x_1 + \dots + w_px_p$$

Ordinary Least Squares